Conversación: etapa 3 (refactoring)

Fecha: 07/11/2026
Integrante: Javier Ovelar
Herramienta: Claude Code (Anthropic), modelo Opus 5.5
Repositorio: TPI_MS2_Mazza_Morales_Ovelar_Paschetta

Se transcriben los mensajes de Javier y las respuestas de Claude. No se incluyen los comandos ni las lecturas de archivos que Claude hizo para responder.

> **Javier**
> siendo javier, estoy en la etapa 3 de refactorizacion, indicame como deberia proceder

Claude

Ya encontré tu rama y cambios a medio hacer. Ahora corro los tests y miro la rama de Gastón para ver si hay conflicto en `dominio.py`.

Ya arrancaste el **paso 1**: el Move Method está hecho y todavía no lo commiteaste. Corrí la suite completa con esos cambios: **67 passed**. Antes de commitear hay algo que cambió desde la consulta anterior.

Situación actual

- Estás en `refactor/javier-estados` con dos archivos modificados sin commitear:
  - `app/dominio.py`: agregaste `TRANSICIONES`, `ESTADOS_CERRADOS` y `transicion_permitida`.
  - `app/routers/estados.py`: `proc_estado` usa `transicion_permitida` y `asignar` usa `ESTADOS_CERRADOS`.
- **La rama de Gastón ya entró a `dev`** (PR #19). Tu rama quedó atrás de `dev`.
- Gastón también agregó contenido **al final** de `app/dominio.py` (`REGLAS_POR_TIPO` y `TIPOS`), en el mismo lugar que vos. Por eso al actualizar va a haber conflicto en ese archivo.

Cómo seguir

1. Actualizar tu rama con `dev` sin perder tus cambios

```
git stash
git merge origin/dev
git stash pop
```

Si `stash pop` avisa que hay conflicto en `app/dominio.py`, abrilo y quedate con **los dos bloques**: primero el de Gastón (`REGLAS_POR_TIPO`, `TIPOS`) y después el tuyo (`TRANSICIONES`, `ESTADOS_CERRADOS`, `transicion_permitida`). Borrá las marcas `<<<<<<<`, `=======` y `>>>>>>>`, y después corré `git add app/dominio.py`.

2. Correr la suite completa

```
.\run.ps1 test
```

Tiene que dar todo en verde, ahora también con los tests que sumó Gastón.

3. Commitear el Move Method solo

```
git add app/dominio.py app/routers/estados.py
git commit -m "refactor: mover las transiciones y los estados cerrados a app/dominio.py"
```

No incluyas el PDF de `anexo/ia/` en este commit. Va en el commit de documentación (paso 6).

4. Avisarle a Bruno `consultas.py:38` todavía tiene escrito `("RES", "RECH", "CANC")`. Ahora puede usar `ESTADOS_CERRADOS`. Ese archivo es de su parte, así que la decisión es suya.

5. Seguir con los demás refactorings, uno por commit, en el orden que ya tenías:

1. ~~Move Method (transiciones y estados cerrados)~~: este es el que acabás de hacer.
2. Leer las columnas por nombre en lugar de `s[5]`, `r[4]`, `est[4]` (hallazgo 9).
3. Extract Method del historial y la notificación en `proc_estado` (hallazgos 8 y 11).
4. Rename de `s`, `r`, `act`, `f` (hallazgo 12).

Después de cada uno: `.\run.ps1 test` y un commit.

6. Documentar: `docs/refactoring/javier_refactoring.md` con el formato de Cristian y Gastón, la captura de la suite en `docs/evidencias/fase3/javier/` y el registro en `docs/uso-ia/javier.md`. El PDF `javier_claude_03_refactoring.pdf` va en este commit.

7. Push y PR hacia `dev`, con Gastón como revisor.

Los hallazgos 1, 4, 5 y 6 siguen afuera: cambian lo que responde el sistema, así que no son refactorings.

No modifiqué ningún archivo. Si querés, te guío con el paso 1 o con el refactoring de columnas por nombre.

Javier

```
run.bat test
========================================= test session starts
========================================= platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-
1.7.0
rootdir: H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 74 items

tests\test_asignacion.py ....                                                        [
5%] tests\test_buscar.py ............
[ 21%] tests\test_detalle.py ......
[ 29%] tests\test_estado.py .....
[ 36%] tests\test_estudiantes.py ..........
```

```
[ 50%] tests\test_reporte.py ..........
[ 63%] tests\test_responsables.py ...
[ 67%] tests\test_solicitudes.py ........................
[100%]

========================================== warnings summary
========================================== .venv\Lib\site-packages\fastapi\testclient.py:1
  H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-
packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================== 74 passed, 1 warning in 10.59s
===================================
```

Claude

Pasaron los 74 tests, incluidos los que sumó Gastón. El merge con `dev` quedó hecho y el conflicto en `app/dominio.py` está bien resuelto: están los dos bloques y no quedaron marcas `<<<<<<<` ni `>>>>>>>`.

Antes de commitear faltan dos cosas que muestra `git status`:

- `app/dominio.py` sigue marcado como **en conflicto** (`UU`). Ya lo arreglaste, pero todavía no le avisaste a git con `git add`.
- El **stash sigue guardado**. Cuando `stash pop` encuentra un conflicto, no lo borra solo.

Siguiente paso

```
git add app/dominio.py app/routers/estados.py
git stash drop
git commit -m "refactor: mover las transiciones y los estados cerrados a app/dominio.py"
```

Con eso queda hecho el refactoring 1, el Move Method que te toca garantizar. Para confirmar, `git log --oneline -1` tiene que mostrar ese commit.

Igual que antes, dejá el PDF de `anexo/ia/` afuera de este commit.

Después seguís con el refactoring 2: leer las columnas por nombre en lugar de `s[5]`, `r[4]` y `est[4]`. Si querés, te guío con ese.

Javier

guiame con el refactoring 2

Claude

El refactoring 1 ya quedó commiteado (`a35ae4e`). Seguimos con el 2.

Refactoring 2: leer las columnas por nombre (hallazgo 9, Magic Numbers)

Qué se hace: se reemplazan los índices (`s[5]`, `r[4]`, `est[4]`…) por el nombre de la columna. Cristian y Gastón lo resolvieron igual. Por ejemplo, en `app/routers/solicitudes.py:75`:

```
c.row_factory = sqlite3.Row
```

Con esa línea, cada fila se puede leer por índice **o** por nombre, así que el comportamiento no cambia.

Qué número es cada columna (sale de `app/db.py`):

Tabla	Índice → nombre
`solicitudes` (`s`)	1 `estudiante_id`, 5 `estado`, 6 `responsable_id`, 7 `area`
`responsables` (`r`)	0 `id`, 1 `nombre`, 2 `area`, 4 `activo`
`estudiantes` (`est`)	4 `email`

Cambios en `app/routers/estados.py`

1. En `proc_estado`, debajo de `c = conn.cursor()` (línea 17), agregar:

```
    c.row_factory = sqlite3.Row
```

Después, reemplazar:

Línea	Antes	Después
23	`act = s[5]`	`act = s["estado"]`
27	`s[6] is None`	`s["responsable_id"] is None`
39	`(s[1],)`	`(s["estudiante_id"],)`
43	`est[4]`	`est["email"]`

2. En `asignar`, debajo de `c = conn.cursor()` (línea 52), agregar la misma línea:

```
    c.row_factory = sqlite3.Row
```

Después, reemplazar:

Línea	Antes	Después
57	`s[5] in ESTADOS_CERRADOS`	`s["estado"] in ESTADOS_CERRADOS`
63	`r[4] == 0`	`r["activo"] == 0`
65	`r[2] != s[7]`	`r["area"] != s["area"]`
66	`... al area " + s[7]`	`... al area " + s["area"]`
69	`nuevo_estado = s[5]`	`nuevo_estado = s["estado"]`

70	`if s[5] == "PEND":`	`if s["estado"] == "PEND":`
73	`(r[0], nuevo_estado, f, id)`	`(r["id"], nuevo_estado, f, id)`
74	`nuevo_estado != s[5]`	`nuevo_estado != s["estado"]`
76	`(id, s[5], ..., "asignada a " + r[1])`	`(id, s["estado"], ..., "asignada a " + r["nombre"])`
78	`"responsable_id": r[0], "responsable": r[1]`	`"responsable_id": r["id"], "responsable": r["nombre"]`

Los nombres de variables (`s`, `r`, `act`, `f`) quedan como están, porque cambiarlos es el refactoring 4. `data["comentario"]` en la línea 29 tampoco se toca: es el body que llega en el pedido, no una fila de la base.

Verificar y commitear

1. Buscá que no haya quedado ningún índice en el archivo:

   ```
   Select-String -Path app\routers\estados.py -Pattern '\b(s|r|est)\[\d'
   ```

   No tiene que mostrar nada.

2. Corré la suite completa con `.\run.bat test`. Tienen que pasar los 74.
3. Commiteá:

   ```
   git add app/routers/estados.py
   git commit -m "refactor: leer las columnas por nombre en proc_estado y asignar"
   ```

Para tu documentación: este refactoring resuelve el smell *Magic Numbers* (hallazgo 9) y aplica el principio de Clean Code de nombres expresivos.

Cuando termines, pegame la salida de los tests y seguimos con el 3 (Extract Method).

Javier

```
Select-String -Path app\routers\estados.py -Pattern '\b(s|r|est)\[\d'

app\routers\estados.py:75:              (r[0], nuevo_estado, f, id))
app\routers\estados.py:76:    if nuevo_estado != s[5]:
```

Claude

Te faltan esas dos líneas, y además hay un error en la **línea 78** que el buscador no detecta. Es culpa mía: en la tabla abrevié esa línea con `...` y quedó copiado literal en el código.

Las líneas se corrieron dos lugares porque agregaste los `row_factory`. Por eso los números ya no coinciden con los de mi tabla.

Corregir en `asignar`

Línea 75:

```
            (r[0], nuevo_estado, f, id))
```

→

```
            (r["id"], nuevo_estado, f, id))
```

Línea 76:

```
    if nuevo_estado != s[5]:
```

→

```
    if nuevo_estado != s["estado"]:
```

Línea 78: tiene que quedar completa, sin los `...`:

```
                (id, s["estado"], nuevo_estado, f, "asignada a " + r["nombre"]))
```

Después

1. Volvé a correr el `Select-String`. No tiene que mostrar nada.
2. Corré `.\run.bat test`. Tienen que pasar los 74. Con los `...` en la línea 78, al menos `test_asignacion.py` fallaría o guardaría un historial incorrecto.
3. Commiteá:

   ```
   git add app/routers/estados.py
   git commit -m "refactor: leer las columnas por nombre en proc_estado y asignar"
   ```

Cuando lo tengas, pegame la salida de los tests.

Javier

```
λ run.bat test
ImportError while loading conftest
'H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\tests\conftest.py'.
tests\conftest.py:11: in <module>
    from main import crear_app
main.py:5: in <module>
    from app.routers import consultas, estados, estudiantes, responsables, solicitudes
E     File "H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\app\routers\estados.py",
line 28
E       if nuevo == "ASIG" and s["responsable_id"] is None
E                                                       ^
E   SyntaxError: expected ':'
Presione una tecla para continuar .
```

Claude

Revisé el archivo completo y hay **tres** errores, no solo el que marca Python. Los otros dos no los avisa porque no son de sintaxis: aparecerían recién al ejecutar o en los tests.

Línea 28: falta el `:` del final.

```
    if nuevo == "ASIG" and s["responsable_id"] is None:
```

Línea 40: quedó un `s[(...)]` envuelto. Tiene que quedar así:

```
    c.execute("SELECT * FROM estudiantes WHERE id = ?", (s["estudiante_id"],))
```

Línea 78: faltan `nuevo_estado, f` (eran los que reemplazaban los `...`). La consulta pide 5 valores y hay 3.

```
                  (id, s["estado"], nuevo_estado, f, "asignada a " + r["nombre"]))
```

El resto del archivo está bien.

Cuando lo corrijas:

1. `.\run.bat test`: tienen que pasar los 74.
2. Commiteá:

   ```
   git add app/routers/estados.py
   git commit -m "refactor: leer las columnas por nombre en proc_estado y asignar"
   ```

Pegame la salida y seguimos.

Javier

```
λ run.bat test
========================================= test session starts
========================================= platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-
1.7.0
rootdir: H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 74 items

tests\test_asignacion.py ....                                                          [
5%] tests\test_buscar.py .
```

Claude

La salida que pegaste está cortada. Corro la suite acá para ver el resultado completo.

La salida que pegaste estaba cortada, así que corrí la suite completa acá: pasaron los 74 tests. Las tres líneas quedaron bien corregidas.

Todavía no está commiteado. Hacelo ahora:

```
git add app/routers/estados.py
git commit -m "refactor: leer las columnas por nombre en proc_estado y asignar"
```

Con eso cerrás el refactoring 2. Avisame y seguimos con el 3: sacar a funciones propias el registro en historial y la notificación de `proc_estado` (Extract Method, hallazgos 8 y 11).

> **Javier**
> ya esta el add y el commit

Claude

El commit quedó registrado (`4a6b960`). Vamos con el 3.

Refactoring 3: Extract Method del historial y la notificación (hallazgos 8 y 11)

Se sigue el mismo criterio que usó Gastón en `app/routers/solicitudes.py:56-69` (`registrar_alta_en_historial` y `notificar_alta`): cada responsabilidad pasa a una función propia, y el endpoint las llama.

Hay dos detalles para que el comportamiento no cambie:

- **El historial no hace `commit` adentro.** El `conn.commit()` queda en cada endpoint, como está ahora. Así la misma función sirve para `proc_estado` y para `asignar`, que hoy repiten el mismo `INSERT`.
- **La notificación recibe el estudiante entero, no el email.** Hoy, si el estudiante no existe, `est["email"]` falla *dentro* del `try` y el error se ignora. Si se leyera el email afuera, el error saldría como un 500.

1. Agregar las dos funciones

Van en `app/routers/estados.py`, debajo de `router = APIRouter()` y antes de `@router.put("/solicitudes/{id}/estado")`:

```
def registrar_en_historial(conn, solicitud_id, estado_ant, estado_nuevo, fecha, comentario):
    """Agrega al historial el paso de la solicitud de un estado a otro."""
    conn.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (?,?,?,?,?)",
                 (solicitud_id, estado_ant, estado_nuevo, fecha, comentario))


def notificar_cambio_estado(estudiante, solicitud_id, estado, fecha):
    """Deja en notificaciones.log el aviso al estudiante del cambio de estado."""
    try:
        with open("notificaciones.log", "a") as arch:
```

```
            arch.write(fecha + " - Se envio mail a " + estudiante["email"] + ": su solicitud " +
str(solicitud_id) + " paso a " + estado + "\n")
    except:
        pass
```

Dejá dos líneas en blanco entre cada función, como en el resto del archivo.

2. En `proc_estado`, reemplazar desde `f = datetime.now()...` hasta el `except: pass`

Antes:

```
    f = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    c.execute("UPDATE solicitudes SET estado = ?, fecha_mod = ? WHERE id = ?", (nuevo, f, id))
    c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (?,?,?,?,?)",
              (id, act, nuevo, f, data.get("comentario", "")))
    conn.commit()

    # notificar al estudiante
    c.execute("SELECT * FROM estudiantes WHERE id = ?", (s["estudiante_id"],))
    est = c.fetchone()
    try:
        with open("notificaciones.log", "a") as arch:
            arch.write(f + " - Se envio mail a " + est["email"] + ": su solicitud " + str(id) + "
paso a " + nuevo + "\n")
    except:
        pass
```

Después:

```
    f = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    c.execute("UPDATE solicitudes SET estado = ?, fecha_mod = ? WHERE id = ?", (nuevo, f, id))
    registrar_en_historial(conn, id, act, nuevo, f, data.get("comentario", ""))
    conn.commit()

    c.execute("SELECT * FROM estudiantes WHERE id = ?", (s["estudiante_id"],))
    est = c.fetchone()
    notificar_cambio_estado(est, id, nuevo, f)
```

El comentario `# notificar al estudiante` se va porque ahora lo dice el nombre de la función. El `return` del final no se toca.

3. En `asignar`, reemplazar el `INSERT` del historial

Antes:

```
    if nuevo_estado != s["estado"]:
        c.execute("INSERT INTO historial (solicitud_id, estado_ant, estado_nuevo, fecha, comentario)
VALUES (?,?,?,?,?)",
                  (id, s["estado"], nuevo_estado, f, "asignada a " + r["nombre"]))
```

Después:

```
    if nuevo_estado != s["estado"]:
        registrar_en_historial(conn, id, s["estado"], nuevo_estado, f, "asignada a " + r["nombre"])
```

Qué no se toca en este paso

- El `except:` sin tipo es el hallazgo 7, aparte.
- Los nombres `s`, `r`, `act` y `f` se cambian en el refactoring 4.
- `solicitudes.py` es de Gastón, así que su historial y su notificación quedan como están.

4. Verificar y commitear

```
.\run.bat test
git add app/routers/estados.py
git commit -m "refactor: extraer el registro en historial y la notificacion de proc_estado y
asignar"
```

Tienen que pasar los 74.

Para tu documentación: resuelve *Duplicated Code* (hallazgo 8, el `INSERT` del historial estaba en las dos funciones) y *Long Method* (hallazgo 11). La técnica es Extract Method y los principios que mejora son SRP y DRY.

Pegame la salida cuando lo tengas. Si se corta como la vez anterior, avisame y la corro yo.

Javier

```
λ run.bat test
========================================= test session starts
========================================= platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-
1.7.0
rootdir: H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 74 items

tests\test_asignacion.py ....                                                            [
5%] tests\test_buscar.py ............
[ 21%] tests\test_detalle.py ......
[ 29%] tests\test_estado.py .....
[ 36%] tests\test_estudiantes.py ..........
[ 50%] tests\test_reporte.py ..........
[ 63%] tests\test_responsables.py ...
[ 67%] tests\test_solicitudes.py ........................
[100%]

========================================= warnings summary
========================================== .venv\Lib\site-packages\fastapi\testclient.py:1
  H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-
packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=================================== 74 passed, 1 warning in 5.80s
===================================
```

Claude

Pasaron los 74 tests y revisé el archivo: el código quedó bien en las dos funciones nuevas, en `proc_estado` y en `asignar`.

Todavía falta el commit:

```
git add app/routers/estados.py
git commit -m "refactor: extraer el registro en historial y la notificacion de proc_estado y
asignar"
```

Con eso cerrás el refactoring 3. El que sigue es el **4: Rename** (hallazgo 12), cambiar los nombres `s`, `r`, `act` y `f` por nombres que digan qué contienen. Avisame cuando hayas commiteado y te guío con ese.

Javier

listo

Claude

El commit quedó registrado (`4c1e380`). Vamos con el último.

Refactoring 4: nombres expresivos (hallazgo 12)

Se cambian solo los nombres de las variables locales. No cambia ninguna lógica:

Antes	Después	Dónde
`s`	`solicitud`	las dos funciones
`r`	`responsable`	`asignar`
`act`	`estado_actual`	`proc_estado`
`f`	`fecha`	las dos funciones
`est`	`estudiante`	`proc_estado`

`c`, `data` y `nuevo` quedan igual. Gastón también dejó `c` en `registrar`.

Para no repetir los errores de copia de la otra vez, abajo van las dos funciones completas. Reemplazá cada una entera, desde el `@router.put` hasta el `return`.

`proc_estado`

```
@router.put("/solicitudes/{id}/estado")
def proc_estado(id: int, data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
    c = conn.cursor()
    c.row_factory = sqlite3.Row
    c.execute("SELECT * FROM solicitudes WHERE id = ?", (id,))
    solicitud = c.fetchone()
    if solicitud is None:
        raise HTTPException(status_code=404, detail="no existe")
    nuevo = data.get("estado")
    estado_actual = solicitud["estado"]

    if not transicion_permitida(estado_actual, nuevo):
        return {"error": "transicion no permitida de " + estado_actual + " a " + str(nuevo)}
    if nuevo == "ASIG" and solicitud["responsable_id"] is None:
        return {"error": "no tiene responsable"}
    if nuevo == "RECH" and ("comentario" not in data or data["comentario"] == ""):
        return {"error": "para rechazar hay que poner motivo"}

    fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    c.execute("UPDATE solicitudes SET estado = ?, fecha_mod = ? WHERE id = ?", (nuevo, fecha, id))
    registrar_en_historial(conn, id, estado_actual, nuevo, fecha, data.get("comentario", ""))
    conn.commit()

    c.execute("SELECT * FROM estudiantes WHERE id = ?", (solicitud["estudiante_id"],))
    estudiante = c.fetchone()
    notificar_cambio_estado(estudiante, id, nuevo, fecha)

    return {"id": id, "estado": nuevo, "estado_anterior": estado_actual}
```

asignar

```
@router.put("/solicitudes/{id}/responsable")
def asignar(id: int, data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
    c = conn.cursor()
    c.row_factory = sqlite3.Row
    c.execute("SELECT * FROM solicitudes WHERE id = ?", (id,))
    solicitud = c.fetchone()
    if solicitud is None:
        return {"error": "solicitud no existe"}
    if solicitud["estado"] in ESTADOS_CERRADOS:
        return {"error": "la solicitud esta cerrada"}
    c.execute("SELECT * FROM responsables WHERE id = ?", (data.get("responsable_id"),))
    responsable = c.fetchone()
    if responsable is None:
        return {"error": "responsable no existe"}
    if responsable["activo"] == 0:
        return {"error": "responsable inactivo"}
    if responsable["area"] != solicitud["area"]:
        return {"error": "el responsable no pertenece al area " + solicitud["area"]}

    fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    nuevo_estado = solicitud["estado"]
    if solicitud["estado"] == "PEND":
        nuevo_estado = "ASIG"
    c.execute("UPDATE solicitudes SET responsable_id = ?, estado = ?, fecha_mod = ? WHERE id = ?",
              (responsable["id"], nuevo_estado, fecha, id))
```

```
    if nuevo_estado != solicitud["estado"]:
        registrar_en_historial(conn, id, solicitud["estado"], nuevo_estado, fecha, "asignada a " +
responsable["nombre"])
    conn.commit()
    return {"id": id, "responsable_id": responsable["id"], "responsable": responsable["nombre"],
"estado": nuevo_estado}
```

Verificar y commitear

```
.\run.bat test
git add app/routers/estados.py
git commit -m "refactor: nombres expresivos en proc_estado y asignar"
```

Tienen que pasar los 74.

Falta tu decisión sobre una parte del hallazgo 12

La ficha también menciona los nombres de las funciones `proc_estado` y `asignar`, y el parámetro `id`, que tapa la función `id()` de Python. En este paso los dejé como están por dos motivos:

- Cambiar `id` también obliga a cambiar la ruta `{id}` de cada endpoint.
- Gastón dejó `registrar` con su nombre original.

Decidí vos si los incluís en este refactoring o los dejás anotados como mejora futura.

Pegame la salida cuando lo tengas.

> **Javier**
> cuanto mas quedaria despues de esto?

Claude

Después del refactoring 4 ya no quedan cambios de código. Lo que falta de la etapa 3 es documentar y abrir el PR:

1. **Decidir lo pendiente del hallazgo 12:** si cambiás `proc_estado`, `asignar` e `id`, o los dejás como mejora futura.
2. **Documentar**, todo en un solo commit:
   - `docs/refactoring/javier_refactoring.md`, con el formato de Cristian y Gastón: una tabla resumen y, por cada uno de los 4 refactorings, el smell, la técnica, el principio y el código antes y después.
   - La captura de la suite en verde en `docs/evidencias/fase3/javier/`.
   - La sección de la fase 3 en `docs/uso-ia/javier.md`.
   - El PDF de esta conversación en `anexo/ia/`.
3. **Subir la rama y abrir el PR hacia** `dev`, con la plantilla del repo y Gastón como revisor.
4. **Avisarle a Bruno** que `ESTADOS_CERRADOS` ya está en `dominio.py`, por si quiere usarlo en `consultas.py`.

Lo más largo es el documento de refactoring. El resto son unos pocos comandos.

> **Javier**
> no indicaste sobre que trabajar para el ultimo refactor. no hay nombre de archivo

Claude

Tenés razón, no lo puse. El archivo es **`app/routers/estados.py`**, el mismo de los refactorings 2 y 3.

Ahí hay que reemplazar enteras las dos funciones, `proc_estado` y `asignar`, con las versiones que te pasé. Cada una va desde su línea `@router.put(...)` hasta su `return`. Las funciones `registrar_en_historial` y `notificar_cambio_estado`, que están arriba, no se tocan.

> **Javier**

```
λ run.bat test
========================================= test session starts
========================================= platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-
1.7.0
rootdir: H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 74 items

tests\test_asignacion.py ....                                                            [
5%] tests\test_buscar.py ............
[ 21%] tests\test_detalle.py ......
[ 29%] tests\test_estado.py .....
[ 36%] tests\test_estudiantes.py ..........
[ 50%] tests\test_reporte.py ..........
[ 63%] tests\test_responsables.py ...
[ 67%] tests\test_solicitudes.py ........................
[100%]

========================================== warnings summary
========================================== .venv\Lib\site-packages\fastapi\testclient.py:1
  H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-
packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================== 74 passed, 1 warning in 7.55s
===================================
```

Claude

Pasaron los 74 tests y revisé `app/routers/estados.py`: ya no queda ningún `s`, `r`, `act`, `f` ni `est`.

Falta el commit:

```
git add app/routers/estados.py
git commit -m "refactor: nombres expresivos en proc_estado y asignar"
```

Con eso quedan hechos los 4 refactorings de código. Lo siguiente es lo que te pasé antes:

1. Decidir si cambiás `proc_estado`, `asignar` e `id`, o los dejás como mejora futura.
2. Documentar.
3. Abrir el PR.
4. Avisarle a Bruno.

Decime con cuál seguimos.

> **Javier**
> detalla el 1

Claude

Punto 1: qué hacer con lo que queda del hallazgo 12

La ficha (`docs/diagnostico/javier_ficha_smells.md:140-147`) marca, además de las variables que ya cambiaste, dos cosas más:

a) Los nombres de las funciones `proc_estado` y `asignar`

- **Cómo se cambia:** solo el nombre en la línea `def`. Por ejemplo, `proc_estado` → `cambiar_estado` y `asignar` → `asignar_responsable`.
- **Qué afecta:** la URL es la misma y los tests no usan esos nombres (lo busqué: solo aparece `asignar` en un comentario de `tests/test_asignacion.py:36`). Lo único que se ve distinto es el título del endpoint en `/docs`, que FastAPI arma a partir del nombre de la función.
- **Antecedente en el grupo:** Cristian hizo exactamente esto en `responsables.py`. Pasó `nuevoResponsable` → `crear_responsable` y `get_resp` → `listar_responsables`, en el commit `6d9cf25`.
- **Esfuerzo:** 2 líneas.

b) El parámetro `id`

- **Cómo se cambia:** `{id}` → `{solicitud_id}` en las dos rutas, y `id` → `solicitud_id` en cada lugar donde se usa dentro de las dos funciones. Son unos 12 lugares.
- **Qué afecta:** la URL que se llama es la misma. En `/docs` el parámetro aparece como `solicitud_id`.
- **Antecedente en el grupo:** nadie lo cambió. Bruno dejó `/solicitudes/{id}` y `id: int` en `getSolicitud` (`consultas.py:93-94`). Si lo cambiás solo vos, las rutas de solicitudes quedan con criterios distintos.

- **Esfuerzo:** más líneas, y más chances de errores de copia.

Mi recomendación

- **(a) sí:** es mínimo, ya tiene precedente en el grupo y completa el hallazgo.
- **(b) dejarlo como mejora futura** en el documento de refactoring, y anotar que conviene hacerlo junto con `consultas.py` para que todas las rutas queden iguales.

La decisión es tuya. Decime qué elegís: (a), (b), las dos o ninguna. Si incluís algo, te paso los cambios exactos con el archivo y las líneas.

> **Javier**
> hago la a, la b queda como mejora futura

Claude

El commit del refactoring 4 quedó registrado (`d87b2e4`). Ahora cambiás los nombres de las funciones.

Cambios en `app/routers/estados.py`

Línea 29:

```
def proc_estado(id: int, data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
```

→

```
def cambiar_estado(id: int, data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
```

Línea 59:

```
def asignar(id: int, data: dict = Body(...), conn: sqlite3.Connection = Depends(get_db)):
```

→

```
def asignar_responsable(id: int, data: dict = Body(...), conn: sqlite3.Connection =
Depends(get_db)):
```

Solo cambia el nombre. Los parámetros y el resto de cada línea quedan igual.

Verificar y commitear

```
.\run.bat test
git add app/routers/estados.py
git commit -m "refactor: renombrar proc_estado y asignar a cambiar_estado y asignar_responsable"
```

Tienen que pasar los 74.

Como es otro cambio, va en un commit aparte, igual que hizo Cristian con sus endpoints. En la documentación lo podés presentar junto con el refactoring 4, porque los dos resuelven el hallazgo 12. Ahí también anotás el parámetro `id` como mejora futura.

Pegame la salida cuando lo tengas.

Javier

```
λ run.bat test
========================================= test session starts
========================================= platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-
1.7.0
rootdir: H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 74 items

tests\test_asignacion.py ....                                                            [
5%] tests\test_buscar.py ............
[ 21%] tests\test_detalle.py ......
[ 29%] tests\test_estado.py .....
[ 36%] tests\test_estudiantes.py ..........
[ 50%] tests\test_reporte.py ..........
[ 63%] tests\test_responsables.py ...
[ 67%] tests\test_solicitudes.py ........................
[100%]

========================================== warnings summary
========================================== .venv\Lib\site-packages\fastapi\testclient.py:1
  H:\UTNFRSN2026\TPIMS2\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-
packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================== 74 passed, 1 warning in 6.40s
==================================
```

Claude

Pasaron los 74 tests y los dos nombres quedaron bien: `cambiar_estado` y `asignar_responsable`.

Falta el commit:

```
git add app/routers/estados.py
git commit -m "refactor: renombrar proc_estado y asignar a cambiar_estado y asignar_responsable"
```

Con esto terminás todos los cambios de código de la etapa 3. Siguen la documentación, el PR y el aviso a Bruno. Decime con cuál seguimos.

> **Javier**
> sigamos

Claude

Lo que sigue es documentar. Son cuatro cosas, en un solo commit.

1. `docs/refactoring/javier_refactoring.md`

Con la misma estructura que Gastón (`gaston_refactoring.md`) y Cristian:

- **Encabezado:** integrante, alcance (`app/routers/estados.py` y `app/dominio.py`), rama `refactor/javier-estados` y verificación: 74 tests después de cada commit, sin cambiar ningún assert.
- **Tabla resumen.** Va armada con lo que hiciste, siguiendo la numeración de Gastón (G1, G2…) y Cristian (R1…):

#	Refactoring	Técnica	Hallazgos	Principio
J1	Transiciones y estados cerrados a `app/dominio.py`	Move Method	2, 10 (3 en parte)	OCP, DRY
J2	Columnas por nombre	Replace Magic Number with Named Access	9	Nombres claros
J3	Historial y notificación en funciones propias	Extract Method	8, 11	SRP, DRY
J4	Nombres de variables	Rename Variable	12	Nombres claros
J5	Nombres de los endpoints	Rename Method	12	Nombres claros

- **Una sección por refactoring** con el antes y el después del código. El "antes" sale de cada commit con `git show <commit>`.
- **Hallazgos que quedan fuera:**
  - 1, 4, 5 y 6 cambian lo que responde el sistema.
  - 7 (`except:`): Gastón ya lo anotó como "Servicio de notificaciones, con Javier".
  - El parámetro `id` del hallazgo 12 queda como mejora futura, junto con `consultas.py`.
  - El hallazgo 3 queda en parte: `asignar_responsable` sigue pasando PEND → ASIG por su cuenta, sin usar `transicion_permitida`.
  - El hallazgo 8 queda en parte: el historial y la notificación de `registrar` (Gastón) siguen aparte.

2. Captura de la suite en verde

`docs/evidencias/fase3/javier/javier_fase3_suite_completa.png`, con la salida de `.\run.bat test` mostrando los 74 passed.

3. `docs/uso-ia/javier.md`

Agregar la fila de **Refactoring (Fase 3)** en la tabla y un párrafo en "Detalle". Va con el grado de intervención real: la IA propuso los cambios y el código, y vos los aplicaste, corriste los tests e hiciste los commits.

4. `anexo/ia/`

El PDF de esta conversación, además del `javier_claude_03_refactoring.pdf` que ya está.

Antes de seguir necesito saber una cosa: **¿el documento de refactoring lo redacto yo o lo escribís vos con mi guía?** Cambia lo que tenés que declarar en el registro de IA, y no voy a crear el archivo sin que me lo pidas.

> **Javier**
> Genera la documentacion asi la leo y la controlo

Claude

Corrijo dos frases que no están respaldadas: una afirmación sobre el hallazgo 3 que deduje sin comprobar, y "avisado a Bruno", que todavía no pasó.

Ahora agrego la Fase 3 al registro de uso de IA.

Generé dos archivos para que los revises. No commiteé nada.

1. `docs/refactoring/javier_refactoring.md` (nuevo)

- Tiene el encabezado, la tabla resumen J1–J5 y una sección por refactoring. El código "antes y después" lo saqué de los diffs de tus cinco commits.
- Al final están la tabla de hallazgos que quedan fuera y la verificación.
- En "Dónde va" usé los códigos que Gastón ya puso en su documento (C3, C5, G4, G5, G7).

2. `docs/uso-ia/javier.md` (modificado)

- Agregué la fila de la Fase 3 en la tabla, con grado de intervención **Alto**, porque la IA propuso el código de cada refactoring y redactó el documento.
- Agregué un párrafo en "Detalle".

Lo que tenés que controlar especialmente

- **Grado de intervención "Alto":** fijate si coincide con cómo lo ves vos.
- **Evidencia en el anexo:** puse `javier_claude_03_refactoring.pdf`, pero ese PDF tiene solo la primera consulta de los pasos. El párrafo dice que la IA "convirtió la conversación en el PDF del anexo". Si vas a exportar esta conversación, hay que agregar su nombre de archivo en la fila. Si no, hay que sacar esa frase.

- **Hallazgo 10:** dice "Avisar a Bruno". Cuando le avises, se puede cambiar a "Avisado".

Lo que falta y tenés que hacer vos

- La captura de la suite en verde: `docs/evidencias/fase3/javier/javier_fase3_suite_completa.png`. El documento ya la nombra así.
- Cuando lo apruebes, el commit:

```
git add docs/refactoring/javier_refactoring.md docs/uso-ia/javier.md
docs/evidencias/fase3/javier anexo/ia
git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
```

> **Javier**
> El grado de internvencio es medio, si bien sugeriste cambios, los mismos se leyeron y luego se aplicaron, para ver si estaban en concordancia con lo solicitado

Claude

Cambié el grado de intervención de la Fase 3 a **Medio** en `docs/uso-ia/javier.md`. También agregué que leíste cada propuesta y controlaste que estuviera en concordancia con lo solicitado antes de aplicarla, en dos lugares:

- **En la fila de la tabla:** después de "...y el código de cada refactoring", sigue "Leí cada propuesta y controlé que estuviera en concordancia con lo solicitado antes de aplicarla".
- **En el párrafo de "Detalle":** "Leí cada propuesta, controlé que estuviera en concordancia con lo solicitado, apliqué el cambio, corrí la suite...".

Lo demás del archivo quedó como estaba. Sigue pendiente lo del PDF del anexo que te marqué antes: si exportás esta conversación, hay que agregar su nombre de archivo en la fila; si no, hay que sacar la frase "convirtió la conversación en el PDF del anexo".

> **Javier**
> que mas falta?

Claude

Para cerrar la etapa 3 te faltan estos pasos, en este orden:

1. **Revisar `docs/refactoring/javier_refactoring.md`**: todavía no me dijiste si lo controlaste.
2. **Resolver la evidencia del anexo.** El registro de IA dice que la IA convirtió la conversación en el PDF del anexo. El único PDF de la fase 3 es `javier_claude_03_refactoring.pdf`, que tiene solo la primera consulta. Tenés dos opciones:
   - Exportar esta conversación, guardarla en `anexo/ia/` y agregar su nombre en la fila.
   - Sacar esa frase del registro.

3. **Captura de la suite en verde.** La carpeta `docs/evidencias/fase3/javier/` todavía no existe. Creala y guardá ahí `javier_fase3_suite_completa.png`, con la salida de `.\run.bat test` mostrando los 74 passed.
4. **Commit de documentación:**

```
 git add docs/refactoring/javier_refactoring.md docs/uso-ia/javier.md
docs/evidencias/fase3/javier anexo/ia
 git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
```

5. **Subir la rama.** Todavía no está en GitHub:

```
 git push -u origin refactor/javier-estados
```

6. **Abrir el PR hacia** `dev` en GitHub, con la plantilla `.github/pull_request_template.md`, y asignar a Gastón como revisor.
7. **Avisarle a Bruno** que `ESTADOS_CERRADOS` está en `app/dominio.py`, por si quiere usarlo en `consultas.py:38`. Cuando le avises, en tu documento cambiá "Avisar a Bruno" por "Avisado a Bruno".

Después, en la Fase 4, Gastón revisa tu PR, vos atendés sus comentarios y revisás el PR de Bruno.

> **Javier**
> donde se guarda la captura?

Claude

La captura va en:

```
docs/evidencias/fase3/javier/javier_fase3_suite_completa.png
```

La carpeta `javier` todavía no existe, así que primero hay que crearla:

```
New-Item -ItemType Directory -Force docs\evidencias\fase3\javier
```

El nombre del archivo tiene que ser exactamente `javier_fase3_suite_completa.png`, porque así lo cita `docs/refactoring/javier_refactoring.md`. Sigue el mismo formato que la de Gastón (`gaston_fase3_suite_completa.png`).

> **Javier**
> uso CMD no powershell. anotate eso

Claude

Anotado en la memoria: de ahora en adelante te paso los comandos en sintaxis de CMD.

En CMD, para crear la carpeta:

```
mkdir docs\evidencias\fase3\javier
```

Después guardás ahí la captura como `javier_fase3_suite_completa.png`.

> **Javier**
> ya esta guardada

Claude

La captura quedó bien guardada en `docs/evidencias/fase3/javier/javier_fase3_suite_completa.png`. Se ve la rama `refactor/javier-estados`, `run.bat test` y **74 passed**.

Antes del commit quedan pendientes dos cosas:

1. **Revisar `docs/refactoring/javier_refactoring.md`**, si todavía no lo controlaste.
2. **Resolver la evidencia del anexo.** El registro de IA dice que la IA convirtió la conversación en el PDF del anexo, pero `javier_claude_03_refactoring.pdf` tiene solo la primera consulta. Exportá esta conversación a `anexo/ia/` y agregá su nombre en la fila, o sacá esa frase.

Cuando estén resueltas, hacé el commit y subí la rama:

```
git add docs\refactoring\javier_refactoring.md docs\uso-ia\javier.md docs\evidencias\fase3\javier
anexo\ia
git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
git push -u origin refactor/javier-estados
```

Después abrís el PR hacia `dev` con Gastón como revisor, y le avisás a Bruno lo de `ESTADOS_CERRADOS`.

> **Javier**
> el .md esta bien

Claude

Perfecto, entonces el documento de refactoring queda como está.

Lo único que falta antes del commit es resolver la evidencia del anexo. Elegí una de estas dos opciones:

- **Exportar esta conversación** como PDF, guardarla en `anexo\ia\` y pasarme el nombre del archivo. Yo lo agrego en la fila del registro de IA.
- **No exportarla.** En ese caso saco la frase "y convirtió la conversación en el PDF del anexo" del párrafo de la Fase 3.

Decime cuál preferís.

> **Javier**
> la exporto, se llama javier_claude_04_refactoring.pdf

Claude

Agregué `anexo/ia/javier_claude_04_refactoring.pdf` como evidencia en la fila de la Fase 3, junto al 03.

Falta revisar una frase del párrafo de "Detalle": dice que **la IA** "convirtió la conversación en el PDF del anexo", y el 04 lo estás exportando vos. Si lo exportás sin pedírmelo a mí, conviene cambiarla. Decime cómo querés que quede.

Cuando el PDF esté guardado en `anexo\ia\`, el commit y el push son:

```
git add docs\refactoring\javier_refactoring.md docs\uso-ia\javier.md docs\evidencias\fase3\javier
anexo\ia
git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
git push -u origin refactor/javier-estados
```

> **Javier**
> cambiala, la exporto yo

Claude

Cambié la frase. Ahora el párrafo de "Detalle" termina así:

> La IA redactó `docs/refactoring/javier_refactoring.md` y yo lo revisé. Exporté la conversación a `anexo/ia/javier_claude_04_refactoring.pdf`.

Sobre el PDF 03 no puse quién lo generó porque no lo sé. Si querés que figure, decime quién lo hizo y lo agrego.

Cuando tengas el PDF 04 guardado en `anexo\ia\`:

```
git add docs\refactoring\javier_refactoring.md docs\uso-ia\javier.md docs\evidencias\fase3\javier
anexo\ia
git commit -m "docs: refactorings de la fase 3 (antes y despues) y registro de uso de IA"
git push -u origin refactor/javier-estados
```

Después abrís el PR hacia `dev` con Gastón como revisor, y le avisás a Bruno lo de `ESTADOS_CERRADOS`.

> **Javier**
> no generaste el pdf de la conversacion?

Claude

No, no lo generé. Me dijiste que lo exportabas vos, así que no creé ningún PDF. En `anexo\ia\` todavía no hay ningún `javier_claude_04_refactoring.pdf`.

Si querés que lo genere yo, puedo armarlo a partir del registro de esta conversación, con el mismo formato que el 03. En ese caso hay que volver a cambiar la frase del registro de IA para que diga que lo generó la IA. Decime cuál de las dos opciones preferís.

> **Javier**
> generalo vos y cambia la frase

Claude

Estoy armando el PDF con el mismo formato que el 03: saco del registro de esta sesión las preguntas y las respuestas, y después lo imprimo con Edge.

El primer intento se colgó por un error en mi script de conversión: hay un caso en el que no avanza de línea. Lo corrijo y lo vuelvo a correr con un tiempo límite.

Ya está generado el PDF. Lo reviso para confirmar que se ve bien.